

cursor-hls-kb

HLS 知識庫系統 — 使用教學指南

Cursor AI × Vitis HLS × Knowledge Base 自動化設計工作流程

系統總覽



集中式知識庫

PostgreSQL 儲存 UG1399 官方規則 (287) + 用戶自定義規則 (100+), 追蹤應用成效



Cursor AI 自動化

透過 Remote SSH 連線 Vitis-HLS 主機, 查詢規則進行設計, 合成成功後自動寫入知識庫



精準回滾機制

_rollback_info 元數據支援 iteration / project 級精確還原 rules_effectiveness 統計



輕量 API 存取

FastAPI (port 8000) 提供規則查詢、迭代記錄、合成結果等端點, 內網直接呼叫

→ 詳見 README 「特色」 「概述」 章節

系統架構與機器配置

HLS01

知識庫主機

PostgreSQL + FastAPI
管理員安裝設置

HLS02

Vitis-HLS 主機

使用者透過 Cursor
代理 HLS 設計

HLS03

Vitis-HLS 主機

使用者透過 Cursor
代理 HLS 設計

存取方式

1. Cursor AI Agent → SSH → Vitis-HLS 主機 → 內網 → FastAPI (port 8000) 讀寫知識庫
2. DBeaver → SSH Tunnel / 直連 → PostgreSQL (port 5432) — 管理員讀寫、使用者唯讀

→ 詳見 README 「系統架構」 「機器配置」 章節

管理員安裝設置 (HLS01)

1

環境準備

Ubuntu 22.04+, 執行 `env_setup.sh` 安裝 Docker / Python3 / pip3 等

2

修改環境變數

`setup.sh` 頂部修改 `DB_ADMIN` / `DB_ADMIN_PASS` (首次使用前必改)

3

執行 `setup.sh`

自動建立 DB schema、容器、匯入 391 條規則 (287 官方 + 104 自定義)

4

驗證

`curl http://localhost:8000/health` 確認 API 正常, 並驗證規則數量

注意: `DB_USER` 不可設為 `user` (PostgreSQL 保留字); 預設 `hls_user` 即可

→ 詳見 README 「管理員快速開始」 「前置條件」 「初始化系統」 章節

資料庫工具 — 管理員速查 (HLS01)

1 backup_restore.py

備份 / 恢復 完整資料庫

► backup

產生 SQL dump + JSON 元數據至
util/backups/

► list

列出所有現存備份檔與時間戳

► restore <file.sql>

從指定 dump 恢復；會覆蓋當前 DB，
需輸入 yes 確認

2 reset_database.py

清空所有數據（保留 schema）

► --stats

先查看目前各表記錄數，不異動資料庫

► (無參數)

清空 projects / iterations /
synthesis_results / rules_effectiveness
等表；schema 保留

3 logger-rollback.py

精準回滾單一迭代（兩階段）

► logger --project --iteration

讀 _rollback_info 產生 YAML 日誌（不
異動 DB）

► rollback --dry-run <log>

預覽：列出將還原的
rules_effectiveness 與刪除項

► rollback <log>

單一交易執行；任一步失敗整體回滾

執行前需具備環境變數：DB_ADMIN=admin · DB_ADMIN_PASS=admin_passwd 由 setup.sh 自動寫入 .env 及 .bashrc，未載入則無法連線資料庫

使用者環境設置

1

編輯 `hls-env.conf`

設定 KB 主機 IP、DB 帳密、Vitis HLS 路徑、FPGA target part

2

執行 `generate-mdc.sh`

讀取 `hls-env.conf`，將 3 個 `.mdc-template` 的 {{變數}} 替換為實際值，輸出至 `.cursor/rules/`

3

安裝 Cursor (Windows)

下載 Cursor 安裝，安裝 Remote - SSH 擴展

4

SSH 連線 Vitis-HLS 主機 (HLS02、HLS03)

開啟 `~/cursorwork/` 為 workspace，確保規則位於 `.cursor/rules/` 下

5

DBeaver 連線知識庫

透過 SSH Tunnel（外網）或直連（內網/VPN）查看資料庫

→ 詳見 README 「使用者工作流程」步驟 1-5

MDC 規則檔結構

三份 .mdc 規則均設為 `alwaysApply: true`，Cursor 對話時自動載入

hls-core.mdc

分類 1-5

1. 文件版本與概述 — API 端點速查、Best Practice
2. 環境配置與驗證 — API URL 自動偵測、Vitis HLS 確認
3. 數據庫連接規範 — `get_database_url()` 函數
4. 知識庫規則系統 — 操作原則、成功判定、管理員專屬
5. 查詢策略 — 雙軌查詢 (Track 1 + Track 2)

hls-code-standards.mdc

分類 6

6. 代碼快照管理 — file header / pragma comment / code_snapshot 格式 / Comment Quality Checklist (16 項)

hls-recording.mdc

分類 7

7. 設計迭代記錄 — Front-Capture / Bind Rules / previous_ii 基線 / complete_iteration API / 重現迭代

→ 由 `generate-mdc.sh + hls-env.conf` 產生；詳見 README 「使用者工作流程」步驟2

任務速查表 — Task → 分類

依任務快速定位到對應的 MDC 分類與 API 端點

當你要做這件事	分類	關鍵內容 / API
確認機器環境、選擇 API URL	分類 2	hostname / IP 自動偵測
設計前查詢規則 + 相似設計	分類 4+5	/api/rules/effective + /api/design/similar
取得 Category 清單（雙軌查詢前）	分類 5	/api/rules/categories
寫 pragma comment + file header	分類 6	三行說明 + 合成成功後填入實測值
確認 code_snapshot 品質	分類 6	Comment Quality Checklist (16 項)
取得 project_id（記錄前必做）	分類 7	/api/projects?type= + project_name 比對
記錄迭代 / 409 / 重現 / 查進度	分類 7	complete_iteration / existing_project_id / code / progress
rollback / 備份 / 重置	分類 4	一律回覆「需系統管理員授權」

Best Practice（強制遵守）

ALWAYS 設計前先查詢 KB 規則與相似設計

ALWAYS 合成後透過 complete_iteration 記錄到 KB

ALWAYS 本地建立 optimization_summary.md

iteration_number 由 API 自動計算

→ 詳見 hls-core.mdc 【分類】1 「Quick Navigation」 「Best Practice」

API 端點速查

依用途分組的 FastAPI 端點清單 · 查詢 (Query) 與 唯一寫入 (Write)

Query · GET 端點 (讀取)		
GET	/health	API 服務與資料庫連線狀態 (healthy / unhealthy)
GET	/api/projects?type={type}	列出專案；記錄前取 project_id 的唯一來源
GET	/api/projects/{project_id}	取得單一專案明細
GET	/api/design/similar?project_type={type}	查詢相似設計 (Track 1) ；按 ii_achieved 升序；可加 target_ii 過濾
GET	/api/design/{iteration_id}/code	取得迭代完整代碼快照 (code_snapshot / cursor_reasoning / pragmas_used)
GET	/api/rules/effective	查詢規則與成效統計 (Track 2) ； rule_type / category / project_type / min_success_rate
GET	/api/rules/categories	取得最新 category 清單 (雙軌查詢前必呼叫)
GET	/api/analytics/project/{project_id}/progress	迭代進度與歷史 (total_iterations、ii_improvement、resource_usage)
Write · POST 端點 (寫入)		
POST	/api/design/complete_iteration	唯一寫入入口：專案建立 + 迭代記錄 + 合成結果 + 規則統計 + rollback 一次完成；iteration_number 由 API 自動計算
POST	/api/projects	建立新專案；同 type 內 name 須唯一；重複 → 409 Conflict + existing_project_id (用該 ID 重新呼叫)

→ 詳見 hls-core.mdc 【分類】 1 「API Endpoints」 「API Response Supplement」

環境驗證與 API 配置

每次 HLS 任務前必須先驗證環境（MDC 分類 2 強制）

API URL 自動偵測

在 KB 主機上：

→ `http://localhost:8000`

在其他機器：

→ `http://{KB_HOST_IP}:8000`

自動檢測以 `hostname` / IP 判斷

環境確認項目

- ✓ 主機名 / IP 地址
- ✓ Vitis HLS 是否可用
- ✓ KB API 連線狀態
- ✓ `curl` / `jq` / `python3` 工具

有 Vitis → 可執行合成

無 Vitis → 提示使用者

DB 連接：Python 腳本禁止硬編碼，必須使用 `get_database_url()` 自動環境檢測函數

備份、回滾、資料管理與存取原則

以下操作均為系統管理員專屬，使用者無權限執行

備份與恢復

備份完整 SQL dump + JSON 元數據
恢復會覆蓋當前 DB

精準回滾

logger 產生 YAML → rollback 執行
兩階段：先預覽 → 後執行
_rollback_info 精確還原統計

重置資料庫

清空數據、保留 schema
完整重建用 setup.sh

知識庫存取原則

操作	執行者
查詢規則 / 設計	Cursor HLS 使用者
寫入新 iteration	Cursor 呼叫 complete_iteration 自動完成
Rollback / 備份 / 重置	系統管理員

→ 詳見 README 「備份與恢復」 「重置資料庫」 「回滾機制」 「知識庫存取原則」 及 hls-core.mdc 【分類】4 「管理員專屬操作」

雙軌查詢策略

每次設計迭代前必須執行，兩軌互補

Track 1：最佳迭代查詢

API: </api/design/similar>

查詢同 project_type 的歷史最佳設計

回傳按 ii_achieved 升序排列

取 approach / pragmas / cursor_reasoning

必要時以 iteration_id 取完整代碼

目的：掌握性能天花板

Track 2：規則效果查詢

API: </api/rules/effective>

取得 Category 清單 (如pipeline, memory, fir, cordic ...)

(可選) Step 0: 設計規格符合應用類 category 優先 (如 fir)

路徑 A: Track 1 有結果 → 推論 category

路徑 B: 無結果 → 依序 fallback 查詢

語意篩選 → 規則效果統計排序 → 比較規則優先權

由 Step 0, 路徑 A / 路徑 B 找到可套用設計規則

目的：精準取得對題規則

雙層規則系統

官方規則 R### (287 條)

來源：UG1399 最佳實踐

範例：R035 Always apply PIPELINE to innermost loop

技術 Category：dataflow, pipeline, memory, interface, optimization, synthesis 等

用戶自定義 P### (100+ 條)

來源：項目優化經驗（可擴展）

範例：P068 合併移位與計算迴圈消除依賴

應用 Category：fir, systolic, cordic 等（人工標注）

規則 Priority 自動設定

Priority	關鍵字	語意
9	always, must, critical, never	強制性
7	do not, avoid, ensure, should	強烈建議
5	consider, may, recommend, prefer	建議性
4	（無符合關鍵字）	預設值

→ 詳見 hls-core.mdc 【分類】4 「知識庫規則系統」及 README 「規則分類機制」

規則分類與人工標注

Category 由人工在規則檔中以 # Category: <名稱> 標注，import_rules.py 直接讀取

類型	Category	說明
技術面向	pipeline, memory, dataflow, interface, optimization, synthesis ...	通用 HLS 最佳實踐，兩份規則檔都有
應用面向	fir, systolic, cordic ...	僅 rules_user_defined.txt（人工標注）

Cursor 使用方式

1. 每次雙軌查詢前先呼叫 /api/rules/categories 取得最新清單
2. 根據使用者輸入推論 category（如「FIR 濾波器，目標 II=1」→ fir + pipeline）
3. API 查詢時自動合併回傳，不分 official / user_defined
4. 新增 category 只需在規則檔標注，import_rules.py 無需修改

→ 詳見 README「人工標注的 Category」「規則分類機制」

設計迭代工作流

設計前

- Front-Capture : 掃描訊息維護 USER_REF_CODE / USER_SPEC
- 雙軌查詢 KB (Track 1 + Track 2)
- Bind Rules : 綁定選定規則至 APPLIED_RULES
- 決定 previous_ii baseline

實作中

- 寫代碼 + pragma comment (不寫 file header)
- 僅允許 .h / .cpp / *_tb.cpp / .inc 四種檔案

合成後

- csim + csynth 皆成功才進行記錄
- 解析報告 → 填入 header comment (實測值)
- Query project_id → complete_iteration
- 更新本地 markdown 文檔備份

|| 未達標 → 以本次結果為基礎，從尚未嘗試的 pragma 組合選取最佳方案，回到「設計前」重新迭代

→ 詳見 hls-core.mdc 【分類】4 + hls-recording.mdc 【分類】7 「主流程」

II 指標與成功判定

Layer A — KB 統計依據

`ii_bneck`

= max(所有 pipeline 迴圈的 Interval)

`success = (current_ii < previous_ii)`

兩者皆為 `ii_bneck`

驅動 `success_count`、`avg_ii_improvement`

`rules_effectiveness` 統計唯一依據

Layer B — 說明補充

`ii_top`

= top-level 函式行的 Interval

寫入 `cursor_reasoning` + 本地文檔

僅 `ii_top` / latency 改善但 `ii_bneck` 未降：

不可將 `success` 設為 `true`

避免混淆 pipeline 規則成效

`previous_ii` 基線：Layer 1 (measured) → Layer 2 (estimated / fallback)

首次迭代由 Cursor 估算理論基線（如 FIR shift_reg[128] → `previous_ii` = 128）；後續迭代取上一輪實測 `ii_bneck`

→ 詳見 `hls-recording.mdc` 【分類】7 「II 指標優先順序」 「`previous_ii` 基線定義」

Code Snapshot 規範

分隔格式

```
// === {filename} ===
```

固定順序：

[.h] → .cpp → *_tb.cpp → [.inc]

.h / .inc 為選擇性（有則收錄）

testbench 必須是通過 csim 的版本

分隔行本身不屬於檔案內容

兩階段寫入

寫代碼時

- pragma comment（每個 #pragma HLS 上方三行）
- Technique / Parameters / Rationale
- 不寫 file header comment

合成成功後

- 一次填入完整 header（實測值）
- Synthesis Result / Resources / Applied Rules
- 所有數值來自 csynth 報告

Comment Quality Checklist (16 項) — 呼叫 complete_iteration 前必須全部通過

迭代記錄 — complete_iteration

唯一寫入入口 — 一次完成專案建立 + 迭代記錄 + 合成結果 + 規則統計 + rollback 資訊

`project_id`

GET /api/projects + project_name 精確比對（唯一來源）

`code_snapshot`

含 header comment（實測值）、通過 Checklist

`synthesis_result`

ii_achieved (= ii_bneck)、latency、timing、resources

`rules_applied[]`

rule_code + previous_ii + current_ii + success

`cursor_reasoning`

ii_bneck / ii_top 前後對照 + 成功判定依據

錯誤處理

409 Conflict → 取回 existing_project_id 重新呼叫 | iteration_number 由 API 自動計算

→ 詳見 `hls-recording.mdc` 【分類】7 「Post-Synthesis」Step 2-4 及 README 「API 端點服務」

重現迭代與障礙排除

重現指定迭代

1. 取得 iteration_id (progress / similar)
2. GET /api/design/{id}/code 取回 code_snapshot
3. 按分隔行還原檔案
4. 自動產生 run_hls.tcl
5. vitis_hls -f run_hls.tcl 執行並驗證

僅用於驗證/學習，不觸發自動記錄

常見障礙排除

DBeaver 連不上？

- SSH 隧道中斷，重建
- docker ps 檢查 PG 容器

Cursor 權限不足？

- 切換至 Agent 模式
- 重新開啟 Remote SSH

API 空的回傳值？

- project_type 大小寫 (fir ≠ FIR)
- min_success_rate 設為 0 (不設成功率門檻)

Demo 範例總覽

6 個實際 Cursor + Vitis HLS 設計過程，展示 KB 在不同設計類型中的運作方式

範例	設計	迭代	模式	重點
Systolic	4x4 output-stationary systolic GEMM	3	A	一次提示即 II=1；5 條 P-rules
Matmul3x3	3x3 矩陣乘法 FIFO 介面	3	A	DATAFLOW 重疊 3 個矩陣
FIR128	128-tap FIR 串流濾波器	3	A	BRAM 循環緩衝 → 2-way 並行 MAC
CORDIC	sin/cos 旋轉 (0 DSP)	3	A	DATAFLOW vs PIPELINE vs 時間共用
DFT-16	16 點離散傅立葉轉換	4	B	II vs timing 設計空間；3 條草案規則
FFT-16	16 點快速傅立葉轉換	2	B	Ping-pong + template banks；4 條草案規則

模式 A — 已有成熟 P-rules

使用者提供規格描述，KB 回傳完整規則，iter 1 即達 II=1

模式 B — P-rules 較少，需架構引導

使用者需在後續迭代提供架構決策，過程中產出草案規則
(待升級為 P####)

各範例 II=1 達成分析

II=1 判定依據為 ii_bneck (pipeline 迴圈 Interval 之 max) , 與 MDC 規則 rules_applied.success 一致

模式 A — iter 1 即達 II=1 (4 個範例)

範例	ii_bneck	iter 1 clock / timing	後續迭代方向
Systolic	1 ✓	10 ns — timing 失敗 (iter 2 → 12 ns)	clock 放寬 → MAC 路徑分解實驗
Matmul3x3	1 ✓	20 ns — timing met	後續優化 latency 和吞吐
FIR128	1 ✓	12 ns — timing met	改串流模式、提升並行度
CORDIC	1 ✓	10 ns — timing met	三種架構風格比較 (全程 0 DSP)

KB 有成熟 P-rules → ii_bneck=1 從首次迭代到位, 後續聚焦 timing / latency / 吞吐 / 面積

模式 B — II=1 需多次迭代或刻意取捨 (2 個範例)

範例	iter 1	最終	過程與原因
FFT-16	2	1 ✓	iter 1 in-place RAW dependency → II=2 ; iter 2 四項結構改造後達 II=1
DFT-16	1	2	iter 1 達 II=1 但 timing 失敗 ; 刻意退讓至 II=2 + 12 ns 換取 timing closure

結論

- FFT — 唯一「透過多次迭代才達成 II=1」的範例 (iter 2 四項結構改造同步解決)
- DFT — 唯一「最終 ii_bneck ≠ 1」的範例 (II=2 是刻意取捨 — II=1 的 timing 無法 close)

操作關鍵提示與 Cursor 行為觀察

確認規則已載入

新環境可輸入「What rules are you following?」或「Summarize your active Cursor rules」確認 .mdc 規則已生效

提示加 follow KB rules

開始設計時建議明確加上「follow KB rules」。若不提示，Cursor 有時會略過 KB 查詢，直接用自身知識設計，不查規則也不記錄

KB 寫入行為不一致

MDC 規定合成成功後自動寫入 KB。實際上 Cursor 有時照做、有時會先詢問使用者確認、有時需使用者主動提醒「save to KB」

指定專案名與迭代

範例常用「save this to KB {project} iter{N}」明確指定，避免 Cursor 建錯專案或跳過記錄

一次調整一個參數

模式 B 中建議每次只改一個變數（II 目標、partition、clock），保留因果可追溯性，利於升級為正式規則

timing 失敗優先放寬 clock

多個範例顯示：放寬 clock（10→12 ns）比改架構更有效。Cursor 有時會過度修改 RTL，可提示先試 clock

→ 綜合 demo 範例的實際操作經驗

Systolic Array — 4×4 矩陣乘法

模式 A | 3 次迭代 | 5 條 P-rules $\rightarrow$ iter 1 即 $II=1$

起始提示

Design a systolic array accelerator for matrix multiplication ($C = A \times B$) using output-stationary architecture, $SIZE \times SIZE$ PEs, AXI stream. Project named Systolic_Demo, please follow KB design rules.

Iteration 1 — 首次設計即達 $II=1$ (timing 失敗 -0.90 ns)

Track 2 回傳 5 條 P-rules : P085 (僅 pipeline t 迴圈)、P084 (PE 陣列 complete partition)、P090 (保守 T 上界)、P094 (A row-major / B col-major)、P047 (MAC 迴圈外 AXI I/O)

架構 : output-stationary wavefront, 16 DSP (每 PE 一個), PIPELINE $II=1$ 於時間維度 t, 空間維度 (i,j) 不 pipeline

結果 : $II=1$ ✓、DSP=16、LUT=4906 — 但 timing 違規 (slack -0.90 ns @ 10 ns clock)

為何 iter 1 就能 $II=1$? KB 規則預先解決了 data dependency (P085 wavefront)、memory port conflict (P084 partition)、stream blocking (P047 phase separation)、loop structure (P085 pipeline on t only) 四個常見失敗原因

Systolic Array — 4×4 矩陣乘法 — 迭代進程

Iter 2 — Clock 放寬 10→12 ns

無架構改動，僅放寬 clock → timing met (slack +0.56 ns)。II=1、DSP=16 不變。

Iter 3 — MAC 路徑分解 (實驗)

提示：「maintaining II=1, try decomposing the MAC critical path to improve timing」

拆分 multiply + accumulate 為兩階段 + BIND_OP。結果：timing met 但 slack 僅 +0.03 ns (反而比 iter 2 差)。結論：此目標元件上，clock 放寬比結構分解更有效。

iter 1→II=1 (P-rules 驅動) → iter 2 clock 放寬解決 timing → iter 3 驗證 MAC 分解效果有限

關鍵規則：P085 P084 P090 P094 P047 (全部 User-defined) + R036

Matmul 3×3 — FIFO 矩陣乘法

模式 A | 3 次迭代 | FIFO → 面積優化 → DATAFLOW 重疊

起始提示

Design a Matrix Multiplier using FIFO, follow the KB rules

Matrix size: A 3×3, B 3×3, Result 3×3

Requirements: read one sample per clock cycle using FIFO interface, minimizing area

Iteration 1 — 面積最小 FIFO 基線 (II=1, DSP=1)

Track 2 回傳：P047 (FIFO write 不可在 MAC 迴圈內)、P035 (僅 local 3×3 buffer)、P039 (stream depth)

架構：四階段序列 (load A → load B → MAC → write C)，僅 innermost loop pipeline II=1

單一 DSP MAC, ap_ctrl_hs 單次呼叫

結果：Latency=73 cycles、DSP=1、LUT=1070 — clock 放寬至 20 ns 後 timing met

P047 防止在 k 累加迴圈中寫 FIFO (會破壞 II=1)；P035 防止不必要的全矩陣複製

Matmul 3×3 — FIFO 矩陣乘法 — 迭代進程

Iter 2 — Panel Load + Flat Output (Latency -41%)

提示：「optimize latency further in the row-stationary direction」

B 先載入 → A 載入 → flat ij 迴圈 (k fully UNROLL, 3 DSP 並行) → Latency 73 → 43 cycles、DSP 1 → 3、LUT -43%

Iter 3 — DATAFLOW 重疊 3 矩陣

使用者提示重疊時序圖。DATAFLOW 拆為 load_panels + compute_all, 內部 FIFO depth=64 (P039)。

單矩陣有效 ~18-24 cycles (重疊後)、DSP=3 不變。P039 防止 FIFO 深度不足導致 deadlock。

correctness first (iter 1) → 單次延遲優化 (iter 2) → 多矩陣吞吐 (iter 3 DATAFLOW)

關鍵規則：P047 P035 P039 (User-defined) + R035/R036 R051 R001

FIR128 — 128-tap 串流濾波器

模式 A | 3 次迭代 | 串流 → 連續模式 → 2-way 並行 MAC

起始提示

Design a 128-tap FIR filter with AXI Streaming interface using a streaming loop architecture, with minimal resource usage. Name FIR128_Demo, follow KB rules.

Iteration 1 — 面積最小串流基線 (II=1, DSP=1, BRAM=2)

Track 2 回傳：P047 (stream I/O 不可在 MAC 迴圈內)、P079 (circular delay line in BRAM)

架構：sample_loop (讀寫 AXI-Stream) + tap_mac (k=0..127, PIPELINE II=1)，單 DSP MAC delay_line[128] 以 circular buffer 存於 BRAM (P079)，避免 128-tap shift register

結果：II=1、Per-sample ~138 cycles、DSP=1、BRAM=2、LUT=295 — 12 ns timing met

P047 是最關鍵規則：自然傾向是把 axis.read() 放在 MAC 迴圈頂端，會立即破壞 II=1

FIR128 — 128-tap 串流濾波器 — 迭代進程

Iter 2 — 連續串流 (ap_ctrl_none)

提示：「outer streaming loop, no NUM_SAMPLES, no fixed packet length」

ap_ctrl_hs → ap_ctrl_none, while(1) 永久運行。性能不變 (~138 cyc/sample)，LUT 略降。

Iter 3 — 2-Way 並行 MAC (吞吐 ×1.9)

提示：「keep ii=1/DSP limit 2/more MAC, try to improve throughput」

k += 2, 兩個 DSP 並行 MAC。delay_line + coeff_rom 改為 complete partition (BRAM→LUT)。

Per-sample 138→72 cycles、DSP=2、BRAM2→0、LUT 259→4199 (full partition 代價)

面積最小 (iter 1) → 真正串流 (iter 2 ap_ctrl_none) → 吞吐翻倍 (iter 3 2-way MAC)

關鍵規則：P047 P079 (User-defined) + R035/R036

CORDIC — sin/cos 旋轉 (0 DSP)

模式 A | 3 次迭代 | 三種架構風格比較 : DATAFLOW vs PIPELINE vs 時間共用

起始提示

Design a CORDIC and name it Cordic_Demo, follow KB rules

Function: rotation mode | Input: angle θ | Output: $\sin(\theta)$, $\cos(\theta)$

Numeric: ap_fixed<16,2> | Algorithm: 16 iterations | Constraints: shift-add only

Iteration 1 — DATAFLOW 全 pipeline (II=1, 0 DSP, BRAM=51)

Track 2 回傳 : P071 (shift-add microrotation, 禁用 DSP)、P072 (atan 預計算表 partition)

架構 : 18 個 DATAFLOW processes + 17 個 hls::stream FIFO, 每 stage 獨立 pipeline

純 shift-add 運算 (P071), 0 DSP — 留給系統中其他加速器

結果 : II=1、Latency=49 cycles、DSP=0、BRAM=51、LUT=5073

BRAM 高是因為 17 個 stream FIFO (depth=32) 各佔 3 BRAM

P071 是架構基石 : CORDIC 數學只需 shift-add, 但沒有明確規則時 HLS 或開發者可能誤用乘法器

CORDIC — sin/cos 旋轉（0 DSP） — 迭代進程

Iter 2 — Register Pipeline (BRAM 51→0, Latency -65%)

提示：「Reduce BRAM by eliminating hls::stream FIFOs, prefer register-based pipeline」
移除 DATAFLOW，改為單一 PIPELINE II=1 + 16 iterations 自動展開為 register stages。
BRAM 51→0、Latency 49→17、FF -69%、LUT -41%。同樣 II=1、0 DSP。

Iter 3 — 時間共用 (LUT -78%, Top II=23)

提示：「reduce LUT by sharing CORDIC stages, keep II=1, allow slight latency increase」
UNROLL factor=1 防止空間展開 → 1 個共用 microrotation 單元重複使用 16 次。
LUT 2977→662、FF 1344→152，但 Top II=1→23（面積 vs 吞吐取捨）

DATAFLOW（吞吐優先）→ 單一 PIPELINE（最佳平衡）→ 時間共用（面積優先），全程 0 DSP

關鍵規則：P071 P072 (User-defined) + R035/R036

DFT-16 — 離散傅立葉轉換

模式 B | 4 次迭代 | II vs timing 設計空間探索 + 3 條草案規則

起始提示

Make a DFT that takes $N=16$ complex inputs and outputs $N=16$ results.
Use fixed-point numbers and a simple double loop with pipeline.
Follow KB rules, save to DFT_Demo project.

Iteration 1 — 最大平行度 (II=1, timing 失敗 -0.84 ns)

Track 2 回傳：僅 P003 (partition 深度控制 load/compute overlap) — P-rules 有限，預期模式 B

架構：complete partition 所有陣列 + PIPELINE II=1 於 inner n loop

結果：II=1 ✓ 但 timing 違規 (slack -0.84 ns @ 10 ns)、LUT=3380、BRAM=0

P003 指導原則：partition 深度決定 II，但 complete partition 在此設計造成 timing 問題

模式 B：P-rules 僅 1 條，無法一次到位，需使用者引導後續迭代方向

DFT-16 — 離散傅立葉轉換 — 迭代進程

Iter 2 — 降低平行度優先 timing (II=4, 15 ns)

提示：「reduce parallel operations, even if it slightly increases latency or II」

移除所有 partition → BRAM 存取, clock 15 ns。II=4、timing met、LUT 3380→526。

草案規則：throughput ↔ timing trade-off — 有時需接受 II 退步換取 timing closure

Iter 3 — 平衡：cyclic/2 + II=2 (timing 仍失敗)

提示：「try cyclic factor=2, target II=2, avoid full partition」

cyclic factor=2 → II=2, 但 timing 仍 -0.84 ns @ 10 ns。草案規則：HLS estimate vs backend flow

Iter 4 — 最終：同架構 + 12 ns → timing met ✓

提示：「use the above configuration, clock fixed at 12 ns」。RTL 不變, 僅放寬 clock。

II=2、timing met (+0.27 ns)。草案規則：clock-only tuning when II unchanged

iter 1 最大 II 但 timing 失敗 → iter 2 犧牲 II 修 timing → iter 3-4 找到平衡點 (II=2, 12 ns)

關鍵規則：P003 (User-defined) + R035/R036 + 3 條草案規則 (待升級 P####)

FFT-16 — 快速傅立葉轉換

模式 B | 2 次迭代 | 4 項結構改造達 II=1 + 4 條草案規則

起始提示

Design a FFT_Demo project following KB rules, an FFT ($N=16$) with complex inputs, multiple stages, fixed-point, pipeline each stage without fully unrolling. Precomputed twiddle factors, good performance without excessive resources.

Iteration 1 — Radix-2 DIT 基線 (II=2, 25 ns timing met)

Track 2 回傳：僅 P035 (no full-buffer memcpy between stages) — P-rules 有限，預期模式 B

架構：4 stage loops 各自 PIPELINE II=1, in-place butterfly 更新

結果：II=2 (in-place RAW dependency 導致 II > 1)、Latency=116、DSP=8
clock 放寬至 25 ns 才 timing met。LUT=3723

In-place 更新模式 ($\text{buf}[i] = f(\text{buf}[i], \text{buf}[j])$) 產生跨迭代 read-after-write dependency → II 無法為 1

FFT-16 — 快速傅立葉轉換 — 迭代進程

Iter 2 — 四項結構改造 → II=1 @ 10 ns

提示：「Break butterfly into multiple pipeline stages, resolve data dependency using buffering」

四項必須同時完成的改造（每項單獨做仍有瓶頸）：

1. Ping-pong double buffer — 消除 in-place RAW dependency（草案規則 1）
2. Template-fixed banks — 編譯期確定 bank → 消除 sparsemux（草案規則 2）
3. ARRAY_PARTITION dim=2 complete — 解決 HLS 200-880 雙寫入衝突（草案規則 3）
4. BIND_OP + split MAC — DSP binding + staged multiply/add → 10 ns timing met（草案規則 4）

結果：II=1 ✓、clock 10 ns timing met、DSP 8→16、LUT 3723→7427

Wall-clock 2× faster（1440 ns vs 2900 ns）

iter 1 基線 II=2（in-place dependency）→ iter 2 四項結構改造同步達成 II=1

關鍵規則：P035（User-defined）+ R035/R036 R074 + 4 條草案規則（待升級 P###）

cursor-hls-kb

Cursor AI × Vitis HLS × Knowledge Base

GitHub: github.com/aicoforge/cursor-hls-kb

License: CC BY-NC 4.0 | Commercial: kevinjan@aicoforge.com